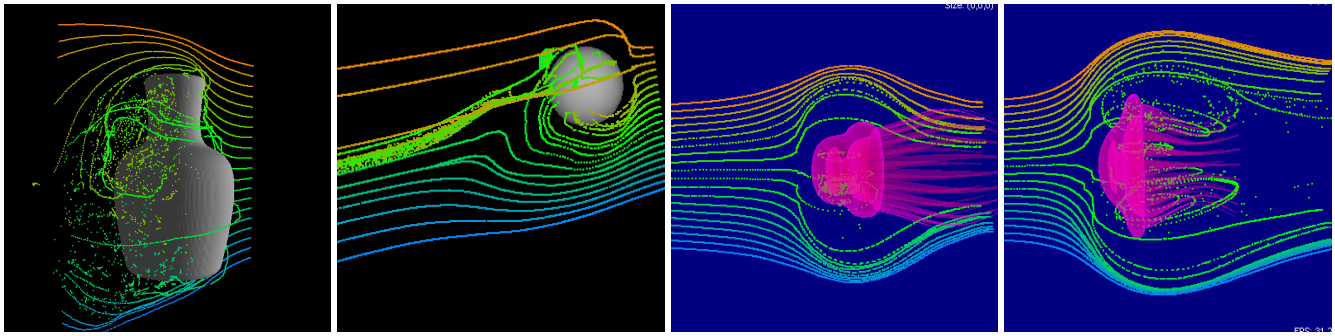
GPU-Based Flow Simulation with Complex Boundaries

Wei Li*
Siemens Corporate Research
Princeton, NJ 08540

Zhe Fan, Xiaoming Wei, and Arie Kaufman†
Stony Brook University
Stony Brook, NY 11790



(a) (b) (c) (d)

Figure 1: Flow fields simulated on a GPU using different obstacles: (a) a static vase; (b) a sphere moving towards the top right; (c) and (d) a jellyfish swimming from right to left. Colored particles, injected at one end through a slit and advected by the flow, depicts the flow field.

Abstract

We present a physically-based flow simulation which supports complex boundary conditions running on the graphics processing unit (GPU). We employ the Lattice Boltzmann Method (LBM), a relatively new discrete-space discrete-time method, for computing the flow field. To handle complex, moving and deformable boundaries, we propose a generic voxelization algorithm of the boundaries using depth peeling, and extend it to a dynamic boundary generation method that converts any geometric boundary to LBM boundary nodes on-the-fly. Our implementation incorporates various optimizations to fully exploit the computation power of the GPU. As a result, the GPU-based simulation can be an order of magnitude faster than the CPU version, while generating simulation results with the same accuracy.

Keywords: GPU, Voxelization, Flow simulation, Boundary condition, LBM

1 Introduction

Physically-based fluid flow simulation is commonplace in many prediction and scientific computation problems, as well as it greatly improves the visual fidelity in computer graphics applications. However, an accurate simulation is computationally expensive. In this paper, we present an accurate yet fast fluid flow simulation method. We choose the Lattice Boltzmann Method (LBM), which yields solutions to the Navier-Stokes (NS) equations with second-order accuracy in space and time, lends itself to interaction with very complex boundaries, and is capable of modeling a wide variety of complex computational fluid dynamics (CFD) problems [Chen and Doolean 1998; Kandhai 1999]. Due to the linear nature of the LBM, it has the potential to run faster than the non-linear Navier-Stokes solvers. Furthermore, our GPU-based LBM simulation computes the exact equations that we have in our CPU-based LBM simulation, but at a much faster speed. Figure 1 shows LBM-based flow simulation results, in which different objects are placed inside the flow field. The position, shape and motion of the objects define the boundary conditions. In Figure 1a, the boundary object is a static vase, while in Figure 1b, it is a sphere that moves towards the top right. Figures 1c and 1d are two snapshots showing a jellyfish swimming from right to left. Note that the jellyfish deforms its body, and so does the liquid-solid boundary. To visualize the flow field, we inject a slice of colored particles and advect them according to the velocity field of the flow. All the computations, the simulation, the generation of the boundaries, the advection, and the rendering of the particles are executed on the GPU in real time. (Please see the videos.)

In the LBM, fluid is considered as a collection of particles that are represented by a particle velocity distribution function at each discrete lattice node. Particles collide with each other and properties associated with the lattice notes are updated at discrete time steps. The rules governing the collisions are designed such that the time-average motion of the particles is consistent with the Navier-Stokes equations. In LBM, the computation of each node at every time step depends solely on the properties of itself and the neighboring nodes at the previous time step. Due to the discrete nature of the LBM and the locality of the computations, it is not difficult to express the LBM equations with GPU programs. However, a straightforward translation results in a very slow implementation. Various algorithm-level and machine-level optimizations are required to actually gain the speed superiority. In this paper, we devise methods for such optimizations and have obtained substantial speedups over CPU implementations.

One of the advantages of the LBM is that it is relatively easy to handle complex, moving and deformable boundaries. However, the boundaries have to be voxelized to discrete boundary nodes that are aligned with the LBM lattice. For moving and deforming boundaries, the voxelization, that primarily involves intersections of the

*e-mail: Wei.Li@scr.siemens.com
†e-mail:{fzhe,wxiaomin,ari}@cs.StonyBrook.edu

boundary polygons with the LBM lattice, ought to be repeated at every time step. Furthermore, for GPU-based LBM and CPU-based voxelization, the transfer of boundary nodes from the main memory to the graphics memory becomes a bottleneck. To address these problems, we propose a fast voxelization algorithm on the GPU and use it to model the interaction of complex obstacles and even living objects with the flow. Another good usage of the voxelization method is collision detection.

The major contributions of this paper are: (a) an accurate fluid flow simulation using the floating point capability of the fragment processor on modern GPUs; (b) optimization techniques throughout the LBM computation stages; (c) a general voxelization algorithm running on the GPU; and (d) extension of the GPU-based voxelization to a boundary condition generation approach that converts boundaries in any representation to LBM boundary nodes on-the-fly. Our approach can be easily adapted for other lattice-based methods. In the remainder of this paper, we first briefly review related work and the theory of the LBM. Then, we describe our major contributions in details, followed by experimental results.

2 Related Work

In recent years, the application of CFD methods by graphics researchers has led to significant developments in the visual simulation of gases and fluids [Foster and Metaxas 1997; Stam 1999; Fedkiw et al. 2001]. These methods solve the NS equations on a discrete voxel grid. However, when the boundary geometry is complex, a relatively large resolution is needed in order to achieve certain accuracy, otherwise grid refinement is necessary. Instead, we adopt the LBM, because it supports complex and dynamic boundaries. Furthermore, unlike the above papers which focus on visual fidelity, our goal is to accelerate the flow simulation on the GPU to attain real-time speed, while maintaining physical accuracy. In our previous works, we have demonstrated LBM-based amorphous phenomena [Wei et al. 2004; Wei et al. 2002], which can be accelerated by the techniques presented in this paper.

Although designed for fast polygon rendering, graphics hardware has been extended to various applications of general-purpose computations, eg. [Hoff et al. 1999; Trendall and Stewart 2000; Larsen and McAllister 2001; Purcell et al. 2002; Lefohn et al. 2003]. Peercy et al. [2000] and later Proudfoot et al. [2001], have developed languages for programmable procedural shading systems as well as compilers that automatically generate instructions corresponding to rendering operations on graphics hardware.

There are a few papers on accelerating flow visualization on graphics hardware [Jobard et al. 2000; Weiskopf et al. 2001]. These techniques are for the visualization of fluid data with given velocity fields. In contrast, our paper focuses on the simulation, specifically the generation of the flow fields. Exploiting such flow visualization techniques, we can map the whole simulation and its visualization onto the graphics hardware without transferring a large amount of data between the main memory and the graphics memory.

Harris et al. [2002; 2003] have implemented on the GPU the coupled map lattice, and have simulated cloud dynamics using partial differential equations. Goodnight et al. [2003] have implemented the multigrid method on the GPU, and have applied it to heat transfer and modeling of fluid mechanics. Boltz et al. [2003] have also developed a GPU-based multigrid solver. In addition, they have proposed a conjugate-gradient solver on the GPU based on a sparse matrix representation, and have applied the solver to the Navier Stokes equations. At the same time, Krüger and Westermann [2003] have presented the GPU implementation of several linear algebra operators. These operators are used to solve the NS equations as well. All these works have similar motivation to that of our paper, in that they translate the computation of fluid dynamics from the CPU to the GPU. With the ever-improving flexibility and programmability of the GPU, this kind of translation is not as difficult as before, whereas finding an optimized translation is much more difficult and is an on-going research topic. In addition to implementing an accelerated LBM that serves as the kernel of various applications involving fluid flow simulation, one of the goals of our work is to accumulate experiences of GPU programming of general computations and strategies of optimizations, which will evolve into an automatic optimization system for various general computation problems.

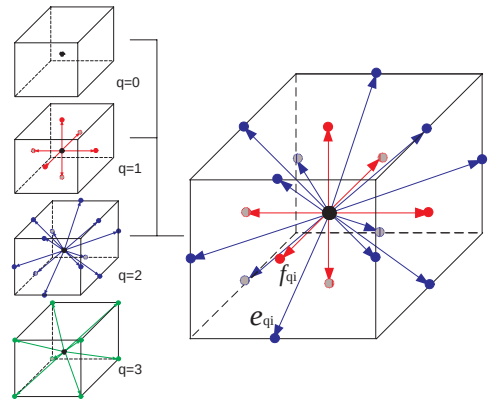


Figure 2: The D3Q19 lattice geometry. On the left side are shown the four sub-lattices that are defined in a 3D lattice. On the right is the combination of sub-lattices 0, 1, and 2. The packet distribution f_{qi} is associated with the link corresponding to the $\mathbf{e}_{qi}$ velocity vector.

In our preliminary work [2003], we have implemented the LBM on previous generation GPU (i.e., Nvidia's Geforce 4) . The work, however, was limited to fixed-point computations on the fragment pipeline, whose flexibility and programmability are very restricted. They had to sacrifice accuracy and used only very simple bounce-back boundary conditions. In contrast, our work takes advantage of the vertex and fragment programs, the floating point texture and frame buffer of the latest generation GPUs, and supports arbitrarily complex boundary conditions.

3 Lattice Boltzmann Method

3.1 Principle

As illustrated in Figure 2, a 3D LBM lattice may be represented as a combination of up to four sub-lattices. For example, the so-called D3Q19 model is a 3D lattice with 19 velocity vectors (including the zero velocity). Stored at each lattice node are the packet distributions f_{qi}, each associated with a velocity vectors $\mathbf{e}_{qi}$, where q identifies a particular sub-lattice and i identifies a particular velocity vector within that sub-lattice. The right part of Figure 2 depicts a single node of the D3Q19 model, while the top-left part of Figure 4 shows four lattice nodes of the D2Q9 (9 packet distributions in 2D space) model. The arrows in the figures represent the $\mathbf{e}_{qi}$ vectors. The macroscopic fluid density, ρ, and velocity, $\mathbf{u}$, are computed from the packet distributions as follows:

$$\rho = \sum_{qi} f_{qi} \qquad \mathbf{u} = \frac{1}{\rho} \sum_{qi} f_{qi} \mathbf{e}_{qi} \tag{1}$$

The LBM updates the packet distribution values at each node based on two rules: collision and streaming (also referred to as propagation). Collision describes the redistribution of packets at each local node. Streaming means the packet distributions move to the nearest

neighbor along the velocity link. These two rules can be described by the following equations:

$$collision : f_{qi}^{new}(\mathbf{x},t) - f_{qi}(\mathbf{x},t) = \Omega_{qi} \quad (2)$$

$$streaming : f_{qi}(\mathbf{x}+\mathbf{e}_{qi},t+1) = f_{qi}^{new}(\mathbf{x},t) \quad (3)$$

where Ω is a general collision operator. Since the components of $\mathbf{e}_{qi}$ can only be chosen from {-1, 0, 1}, the propagation is local.

The collision operator is selected in a way that mass and momentum are conserved locally. The Bhatnager, Gross, Krook (BGK) model is typically used to represent the effect of these collisions as a statistical redistribution of momentum which locally drives the system toward equilibrium [Wolf-Gladrow 2000]. Suppose that there is always a local equilibrium particle distribution f_{qi}^{eq} dependent only on the quantities ρ and $\mathbf{u}$, then the collision step is changed to:

$$f_{qi}^{new}(\mathbf{x},t) - f_{qi}(\mathbf{x},t) = -\frac{1}{\tau}(f_{qi}(\mathbf{x},t) - f_{qi}^{eq}(\rho,\mathbf{u})) \quad (4)$$

where f_{qi}^{eq} is determined by the following equation:

$$f_{qi}^{eq}(\rho,\mathbf{u}) = \rho(A_q + B_q < \mathbf{e}_{qi},\mathbf{u} > + C_q < \mathbf{e}_{qi},\mathbf{u} >^2 + D_q < \mathbf{u},\mathbf{u} >) \quad (5)$$

where $< \mathbf{x},\mathbf{y} >$ denotes the dot product between two vectors $\mathbf{x}$ and $\mathbf{y}$. The constant τ represents the relaxation time scale which determines the viscosity of the flow, while A_q through D_q are constant coefficients specific to the chosen lattice geometry (e.g., D3Q19).

3.2 Boundary Conditions

Boundary conditions in the LBM may take several forms. Those boundary conditions that can be applied to the border of the computation domain include periodic boundary and out-flow boundary. For periodic boundary, the out-going packet distribution wraps around, and re-enters the lattice from the other side. For out-flow conditions, some packet distributions of the boundary nodes propagate outside of the lattice, which can simply be discarded. However, these nodes also expect some packet distributions to propagate inwards from fictitious nodes just outside the boundary. One solution is to copy the distribution of the internal nodes to those boundary nodes [Mei et al. 1999], according to Equations 6, 7, and 8, in which $f(B)_{(i,j,k)}$ is the distribution of a boundary node in the velocity direction of (i,j,k), $f(I)_{(i,j,k)}$ is the distribution of the corresponding internal node, and $i,j,k\epsilon\{-1,0,1\}$. For example, boundary nodes on slice 0 is copied from internal nodes on slice 2. For an outflow face orthogonal to the X axis:

$$f(B)_{(i,j,k)} = f(I)_{(-i,j,k)} \quad (6)$$

For an outflow face orthogonal to the Y axis:

$$f(B)_{(i,j,k)} = f(I)_{(i,-j,k)} \quad (7)$$

For an outflow face orthogonal to the Z axis:

$$f(B)_{(i,j,k)} = f(I)_{(i,j,-k)} \quad (8)$$

For obstacle boundaries, the so called no-slip boundary condition, expresses the requirement that the tangential component of the fluid velocity along the boundary be zero. The simplest implementation is using a bounce-back rule, which assumes that all the boundaries just lie on the lattice nodes and aligned with the velocity links. In this case, the out-going packet distribution value re-enters the lattice at the same node, but associated with the opposite velocity. For boundaries involving complex geometries or those that can move, the bounce-back rule has been substantially improved by

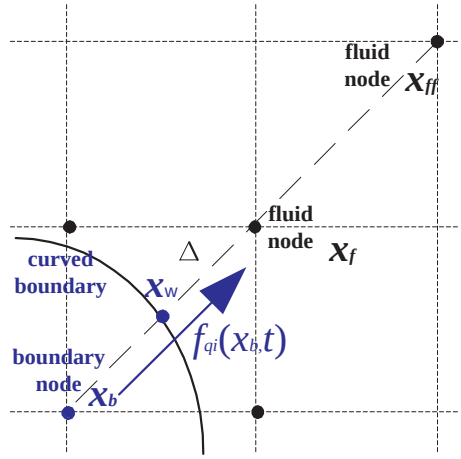


Figure 3: A slice of a regularly spaced lattice and a curved wall boundary.

Mei et al. [Mei et al. 2000]. As indicated in Figure 3, the packet distribution for a node, $\mathbf{x}_f$, in the fluid adjacent to the boundary, is streamed from it neighbors. Thus, a fictitious packet distribution, $f_{qi}(\mathbf{x}_b,t)$, is defined on the node $\mathbf{x}_b$ which lies just inside the object boundary. The fraction of the link that is intersected by the boundary in the fluid region is denoted by $\Delta = |\mathbf{x}_f - \mathbf{x}_w|/|\mathbf{x}_f - \mathbf{x}_b|$. The post-collision value of $f_{qi}(\mathbf{x}_b,t)$ is given by:

$$f_{qi}(\mathbf{x}_b,t) = (1-\chi)f_{qi}(\mathbf{x}_f,t) + \chi f_{qi}^*(\mathbf{x}_b) + 6A_q\rho\mathbf{e}_{qi}\cdot\mathbf{u}_w \quad (9)$$

where,

$$f_{qi}^*(\mathbf{x}_b) = \rho(A_q + B_q\mathbf{e}_{qi}\cdot\mathbf{u}_{bf} + C_q(\mathbf{e}_{qi}\cdot\mathbf{u}_f)^2 - D_q(\mathbf{u}_f)^2) \quad (10)$$

and for $\Delta \geq 1/2$,

$$\mathbf{u}_{bf} = (1-\frac{3}{2\Delta})\mathbf{u}_f + \frac{3}{2\Delta}\mathbf{u}_w \quad \text{and} \quad \chi = \frac{2\Delta-1}{\tau+1/2} \quad (11)$$

while for $\Delta < 1/2$,

$$\mathbf{u}_{bf} = \mathbf{u}_{ff} \quad \text{and} \quad \chi = \frac{2\Delta-1}{\tau-2}. \quad (12)$$

Note that $\mathbf{u}_{bf}$ represents the virtual speed of the boundary node $\mathbf{x}_b$ in terms of the fluid velocity, $\mathbf{u}_f$ at node $\mathbf{x}_f$, $\mathbf{u}_{ff}$ at node $\mathbf{x}_{ff}$, and the boundary velocity, $\mathbf{u}_w$ at $\mathbf{x}_w$. Thus, we see that this rule (which has second order accuracy and good stability) accounts for both arbitrarily shaped and moving boundaries.

4 GPU Based LBM

4.1 Algorithm Overview

To compute the LBM equations on graphics hardware, we divide the LBM lattice and group the packet distributions f_{qi} into arrays according to their velocity vectors. All the packet distributions of the same velocity vector are grouped into the same array, while keeping the neighboring relationship of the original model. Figure 4 shows the division of a 2D model D2Q9 according to its velocity directions. The division of a 3D model is similar. We then store the arrays as 2D textures. For a 2D model, all such arrays are naturally 2D, while for a 3D model, each array forms a volume. The volume is treated as a set of slices and is stored as a stack of 2D textures or a tiled larger texture.

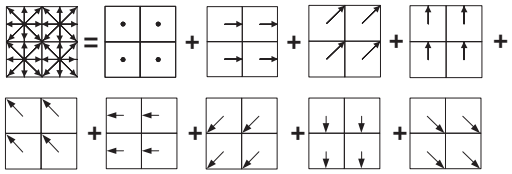


Figure 4: Division of the D2Q9 model, according to its velocity directions.

All the other variables, the density ρ, the velocity $\mathbf{u}$, and the equilibrium distributions f_{qi}^{eq} are stored similarly in 2D textures. We then render quads mapped with those textures, and use fragment programs to compute the LBM equations.

Figure 5 shows the dataflow of the LBM computation on the GPU, where the textures storing lattice properties are represented by the red boxes, while the operations are represented by the blue round boxes. The textures of the packet distributions are the inputs. Density and velocity textures are then dynamically generated from the distribution textures. Next, the equilibrium distribution textures are obtained from the densities and the velocities. According to the collision and the streaming equations, new distributions are computed from the input distributions and the equilibrium distributions. Finally, we apply the boundary and outflow conditions and update the distribution textures. The updated distribution textures are then used as inputs for the next simulation step.

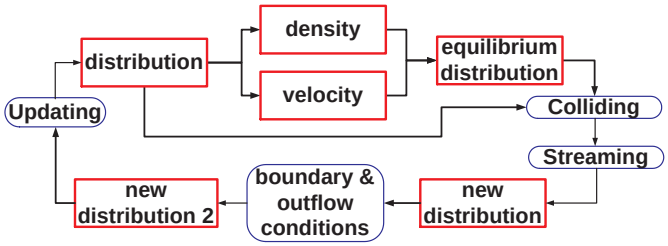


Figure 5: Flow chart of the LBM Computation on the GPU. Red boxes are the textures, while blue round boxes are operations.

The LBM equations are translated to the operations of the fragment pipeline. With the GPUs becoming more and more programmable, a straightforward translation following Figure 5 is not difficult. However, there are usually multiple choices in mapping each equation, and different combinations of the mapping usually result in dramatic difference in performance. In the following sections, we describe various optimization strategies that have been applied to our GPU-based LBM.

4.2 Packing

During the simulation, textures are updated dynamically at every step by copying from or binding to the frame buffer (or the pixel buffer). In the current graphics hardware, it is most efficient to use RGBA textures. Each RGBA texel has four channels, hence can store up to four scalars or a vector with up to four components.

The first optimization we propose is packing different variables into the same texel. For example, we pack four f_{qi}s from different directions into a single RGBA texel. In addition, we attempt to pack together those variables that are involved in the same LBM equations, in order to reduce the number of textures to be activated and the number of texture fetches. This also improves the data locality, as well as the cache coherence of the textures.

Table 1 lists the contents of the textures packed with the variables of the D3Q19 model, including densities, velocities and packet distributions. In texture $\mathbf{u}\rho$, v_x, v_y, and v_z are the three components of the velocity stored in the RGB channels, while the density ρ is stored in the alpha channel. The rows in Table 1 for textures *f0* through *f4* show the packing patterns of both the packet distributions f_{qi} and the equilibrium distributions f_{qi}^{eq}. $f_{(x,y,z)}$ is the distribution in the direction of (x, y, z). Note that we pack distributions of the opposite directions in pairs into the same texture. There are two reasons for that. First, when handling complex or moving boundaries, neighboring distributions at opposite directions are needed to evaluate the effects on the same boundary link. Second, when programming the fragment pipeline, we typically need to pass the corresponding velocity vector $\mathbf{e}_{qi}$ as an argument of the fragment program. When opposite distributions are always neighbors, just two, instead of four, $\mathbf{e}_{qi}$s are needed, while the other two are easily inferred.

Table 1: Packed LBM variables of the D3Q19 model

Texture	R	G	B	A
$\mathbf{u}\rho$	v_x	v_y	v_z	ρ
$f0$	$f_{(1,0,0)}$	$f_{(-1,0,0)}$	$f_{(0,1,0)}$	$f_{(0,-1,0)}$
$f1$	$f_{(1,1,0)}$	$f_{(-1,-1,0)}$	$f_{(1,-1,0)}$	$f_{(-1,1,0)}$
$f2$	$f_{(1,0,1)}$	$f_{(-1,0,-1)}$	$f_{(1,0,-1)}$	$f_{(-1,0,1)}$
$f3$	$f_{(0,1,1)}$	$f_{(0,-1,-1)}$	$f_{(0,1,-1)}$	$f_{(0,-1,1)}$
$f4$	$f_{(0,0,1)}$	$f_{(0,0,-1)}$	$f_{(0,0,0)}$	unused

4.3 Flat Volume

Instead of using a stack of 2D textures to represent a volume, we actually stitch the slices to create a larger 2D texture, that can be considered as a "flat" volume. Similar approaches have been reported in the literature [Li et al. 2003; Harris et al. 2003; Lefohn et al. 2003]. One advantage of the flat volume is the reduced number of texture switching. It also reduces the number of proxy quads. Specifically, a $W \times H \times D$ volume is stored as a $(W*d1) \times (H*d2)$ texture, where $D = d1*d2$. Intuitively, we would choose $d1 = 1$ or $d2 = 1$, which is the simplest for computing the texture coordinates of the flattened texture using the volume coordinates. However, we found that it is much slower than choosing $d1$ to be a factor of D that is the closest to $\sqrt{D}$.

More importantly, using the flat volume is also critical in two stages of our algorithms, particle advection (Section 6) and voxelization and boundary nodes generation (Section 5.1). Without a flat representation, these two will not work. The conversion from the volume coordinates (x, y, z) to the coordinates (u, v) of the flattened texture is as follows:

$$\begin{aligned} u &= (z\%d1) * W + x \\ v &= floor(z/d1) * H + y \end{aligned} \tag{13}$$

where W and H are dimensions of each slice in the volume, and every set of $d1$ slices is tiled in a row along the X direction in the flat volume.

4.4 Streaming

According to Equation 3, each packet distribution having non-zero velocity propagates to a neighboring lattice point at every time step. On the newest generation of GPU, such as Nvidia's GeForce FX, the texture fetching unit can obtain a texel at arbitrary position indicated by texture coordinates fully controlled by the fragment program. If a distribution f is propagated along vector $\mathbf{e}$, we simply fetch from the distribution texture at the position of current lattice

position minus $\mathbf{e}$. Since the four channels are packed with four distributions with different velocity vectors, four fetches are needed for each fragment.

Figure 6 shows as an example the propagation of distribution texture *f1*, which is packed with $f_{(1,1,0)}$, $f_{(-1,-1,0)}$, $f_{(1,-1,0)}$, and $f_{(-1,1,0)}$. The fragment program fetches texels by adding the negative values of the velocity directions to the texture coordinates and extracting the proper color component. For example, the blue channel in texture *f1* is $f_{(1,-1,0)}$. After propagation, the value of the blue channel will come from a texel at relative position $(-1, 1, 0)$ to the fragment position. This is equivalent to translating distribution $f_{(1,-1,0)}$ in the direction of (1,-1,0) by 1.

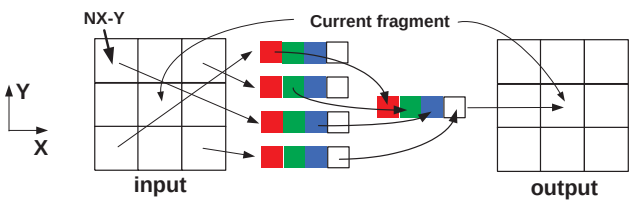


Figure 6: Propagation of distribution texture *f1*, which is packed with $f_{(1,1,0)}$, $f_{(-1,-1,0)}$, $f_{(1,-1,0)}$, and $f_{(-1,1,0)}$.

To propagate using the flat volume, for each channel, we can add the 3D position of the fragment with the negative of the corresponding $\mathbf{e}_{qi}$, then convert it to the texture coordinate of the flat volume according to Equation 13 before fetching. However, this requires Equation 13 be executed four times per fragment. One optimization we apply is to push the coordinates conversion to the vertex level, since for each channel inside a slice, the velocity vectors are the same. We can either assign each vertex of the proxy quad with four texture coordinates containing the converted values, or generate the texture coordinates with a vertex program.

5 GPU-based Boundary Handling

To handle an complex boundary, we need to compute the intersections of the boundary surface with all the LBM lattice links. For a static obstacle, the intersections can be pre-computed. Whereas for either a moving or deformable boundary, the intersection positions change dynamically. The boundary description can be either continuous, such as a polygonal mesh or a higher-order surface, or discrete, such as a volume. No matter what form a boundary is originated from, the handling of the boundary conditions requires discrete boundary nodes aligned with the LBM Lattices. Even if a boundary is already in a volumetric representation, it has to be revoxelized whenever it moves or deforms.

One solution is to compute the intersection and voxelization on the CPU, and then transfer the computed volumetric boundary information from the main memory to the graphics memory. Unfortunately, both the computation and the data transfer are too time-consuming for interactive applications. Naturally, we want to accelerate the volumetric boundary generation on the GPU as well.

In the following sections, we first propose a general-purpose GPU-based voxelization algorithm that converts an arbitrary model to a Cartesian grid volume. Then, we discuss the handling of three different boundary conditions, while focusing on arbitrary complex boundaries that can move and deform. The generation of the boundary nodes of arbitrary boundaries is performed by extending our general-purpose GPU-based voxelization.

5.1 GPU-based Voxelization

An intuitive voxelization approach is the slicing method, which sets the near and far clip planes, so that for each rendering pass, only the geometries falling into the slab between the two clip planes are rendered [Fang and Chen 2000]. This creates one slice of the resulting volume. Then, the clip planes are shifted to generate subsequent slices. Obviously, for this slicing method, the number of rendering passes is the same as the number of the slices in the volume. In most cases, the boundaries are sparse in a volume. Or in other words, only a small percentage of voxels are intersected by the boundary surfaces. There is no need to voxelize the "empty" space that corresponds to non-boundary voxels.

Our GPU-based voxelization avoids the slicing. Instead, we adapt the idea of depth peeling [Everitt 2001] used for order-independent transparency, in which the depth layers in the scene are stripped away with successive passes. In the first rendering pass, the scene is rendered normally, and the layer of the nearest fragments, or equivalently, voxels are obtained. From the second rendering pass, each fragment is compared with a depth texture copied from the depth buffer of the previous pass. A fragment reaches the frame buffer only if its depth value is greater than that of the corresponding pixel in the depth texture, while the ordinary depth test is still enabled. Therefore, the second pass generates the second nearest layer, and so on. The process continues until no fragment is further away than the corresponding pixel in the depth texture, which is best determined with the occlusion query extension that returns the pixel count written to the frame buffer. The number of rendering passes of the depth peeling algorithm is the number of layers plus one, which typically is significantly smaller than the number of slices.

When rendering order-independent transparent objects, all the layer images are blended in depth order. In contrast, in voxelization, we want the layers to be separated, which is similar to a layered depth image [Shade et al. 1998]. Assume that the maximum size along any of the major axes of the object being voxelized is D, we allocate a pixel buffer with width and height of maximum number of layers times D and number of attributes times D, respectively. Then, between different rendering passes, we translate the viewport, so that the layer images do not overlap, but be tiled as tight as possible. The pixels of those images are the attributes of the corresponding voxels. The first attribute has to be the 3D positions, while we may need other attributes depending on the application. As shown in the following pseudo-code, the peeling process is applied three times. Each time the image plane is orthogonal to one of the major axes. That is, we perform the peeling from three orthogonal views to avoid missing voxels. As a result, some of the voxels may be rendered more than once. However, the replication does not affect the final results. The image containing the voxel attributes are then copied to a vertex array whose memory is allocated inside the video memory using extensions, such as Nvidia's PDR and VAR. Note that different type of voxel attributes are copied to different locations inside the vertex array.

```
for each view direction in X, Y, Z do
  layer = 0
  while HasMoreLayer() do
    for each attribute do
      SetViewportOrigin(attrib_id*D, layer*D)
      RenderScene()
    end for
    if HasMoreLayer() then
      UpdateDepthTexture()
    end if
    layer=layer+1
  end while
end for
```

The vertex array is essentially an array of voxel positions plus other attributes, that can generate all the boundary voxels for further processing. We may want to convert the vertex array to a flat volume by simply rendering each vertex as a point of size 1. All the vertices of the array pass through a vertex program that translates each voxel properly according to its z value using equations similar to Equation 13. The frame buffers for the depth peeling are initialized with some large numbers. If a pixel is not covered by any boundary voxels, then the corresponding vertex created from the pixel falls far away from the view frustum, and is clipped.

5.2 Periodic Boundary

In practice, the periodic boundary is actually computed during the propagation. If a periodic boundary face is orthogonal to the X or Y axis, we call it in-slice periodic boundary face, since a distribution on the face is copied to the opposite side of the lattice but stays inside the same XY slice. For in-slice periodic boundary, we simply apply a modulo operation to the texture coordinates by the width or the height of each slice. Whereas for periodic boundary face perpendicular to the Z axis, which we call out-slice periodic boundary, we need to copy distribution textures of one slice to another.

A naive implementation of the in-slice periodic boundary condition is to apply the modulo operation of the texture coordinates of all the distribution texels. However, this can be very slow. For instance, in Nvidia's fragment program, the modulo is simulated by floating point division. Therefore, one optimization we use first propagates without the periodic boundary condition. Then, we draw stripes of single-texel width that only cover the boundary but with the modulo operation. In this way, the cost for computing is negligible, since the periodic boundary nodes only account for a small percentage of the lattice.

5.3 Outflow Boundary

Similar to periodic boundary, an outflow boundary condition is applied by drawing single-texel-wide stripes. According to Equations 6, 7, and 8, nodes at an outflow boundary get their distributions from distributions of internal nodes but with a velocity direction flipped around one of the major axes. Note that when packing the distributions, we guarantee that $f_{(i,j,k)}$ and $f_{(-i,j,k)}$ of the same node are in the same texture, as well as the pairs $f_{(i,j,k)}$ and $f_{(i,-j,k)}$, $f_{(i,j,k)}$ and $f_{(i,j,-k)}$. These can be easily verified from Table 1. Therefore, each boundary distribution texture copies from only one distribution texture. To flip the distributions around the major axes, we utilize the swizzle operator to rearrange the order of the color elements.

5.4 Complex Boundary

To handle a complex boundary, we adopt Mei et al.'s method [Mei et al. 2000] (also Section 3.2), in which the boundary does not necessarily lies on the lattice nodes, and can deform during the simulation. Each boundary link is specified by two nodes $\mathbf{x}_f$, $\mathbf{x}_b$, the fraction of the link that is in fluid Δ, and the moving speed of the wall $\mathbf{u}_w$. We actually deem the regions isolated by the boundary surface as separate fluids. Hence, for each boundary link, the boundary condition affects two distributions, one on each side of the boundary. Besides, the two distributions are in the opposite directions, but co-linear with the link. We refer to them as boundary distributions.

To generate the boundary information, we first create a voxelization of the boundaries using the method described in Section 5.1. Besides the position of the voxels, we also need the wall velocity $\mathbf{u}_w$, as well as the coefficients of polygon plane equations which will be used to compute Δ. That is, we need three attributes in total. To preserve accuracy, we treat these attributes as texture coordinates when rendering the vertex array in the next step.

In practice, we don't explicitly generate the flat volume of the boundary voxels, but combine the generation with the computation of the boundary conditions, by rendering the boundary vertex array directly into the pbuffer containing the propagated new distributions, and applying the fragment program for complex boundary conditions. Note that in most cases, for each boundary link, only one node is covered by a voxel from the generated voxel array. However, we need each boundary node to receive a fragment so that the boundary distributions are updated. Therefore, for each packed distribution texture, we render the voxel array three times. In the first pass, they are rendered normally, covering those voxels in the generated voxel array. In the second pass, we first set the color mask, so that only the R and G channels can be modified. Then, we apply a translation to all the voxels using a vertex program. The translated offset is computed according to the following rule:

- $\mathbf{e}_{qi}$: if $flag_1 * flag_2 > 0$
- $-\mathbf{e}_{qi}$: if $flag_1 * flag_2 < 0$
- 0: if $flag_1 * flag_2 = 0$

where $flag_1 = (pos_x, pos_y, pos_z, 1) \bullet (A, B, C, D)$, $flag_2 = (A, B, C) \bullet \mathbf{e}_{qi}$, and $\bullet$ represents a dot product. (pos_x, pos_y, pos_z) is the 3D position of the voxel without translation. The boundary surface inside the voxel is defined by $Ax + By + Cz + D = 0$, where (A, B, C) is a normalized plane normal pointing from the solid region to the liquid region. $\mathbf{e}_{qi}$ is the velocity vector associated with the distribution in the red channel. The third pass is similar to the second pass, except that this time the B and A channels are modified, and $\mathbf{e}_{qi}$ is the velocity vector corresponding to the blue channel distribution.

In this way, all the boundary nodes are covered by the voxels. We then compute Δ at the beginning of the boundary condition fragment program with the following equations:

$$\Delta' = flag_1/flag_2, \qquad \Delta = 1 - \Delta' \tag{14}$$

The meanings of $flag_1$ and $flag_2$ are the same as before. Note that each channel computes its $flag_1$, $flag_2$, and Δ independent of its own $\mathbf{e}_{qi}$. A distribution is a boundary distribution if only, for the corresponding color channel, $1 \geq \Delta' \geq 0$. If it is not a boundary distribution, the fragment program prevents modifying the corresponding color channel by assigning it the old value.

6 Visualization

To visualize the simulation, we inject particles into the flow field. The positions of the particles are stored in a texture, and are updated by the current velocity field at every time step. The updating is through a fragment program as well. Similar to the generation of boundary voxels, the updated particle positions are then copied to a vertex array residing in the graphics memory for rendering. The whole simulation and rendering cycle is inside the GPU, hence there is no need to transfer large chunks of data between the main memory and the graphics memory. To better display the flow field, particles are arranged into a regular grid before injection and are colored according to the position where they enter the flow field, as shown in Figure 1. Due to the requirement of the vertex array, the total number of particles are constant during the simulation. We use a fragment program to recycle particles that flow out of the border of the field or stay in a zero-velocity-region, and place them back at the inlet. If two particles coincide at exactly the same location at the same time, they will never separate during the advection. Visually, we will see fewer and fewer particles. To avoid this, we add a random offset to each particle when placing it at the inlet.

In 2D LBM, there is only one velocity slice, while in 3D LBM, the velocities form a volume. The advection fragment program fetches the velocity indicated by the current position of the particles. Therefore, the fragment program needs to access either a 3D texture or a 2D texture storing the flat volume. We indeed chose the flat volume storage, which is much faster. Please note that if the velocity is stored as a stack of 2D textures, the advection would be very difficult, if not impossible.

7 Experimental Results

We have experimented with our GPU-based LBM implemented using OpenGL and Cg [Mark et al. 2003] on an Nvidia GeForce FX 5900 Ultra. The graphics board has 256MB 425MHz DDR SDRAM and its core speed is 400MHz. The host computer is a Pentium IV 2.53Ghz with 1GB PC800 RDRAM. All the results related to the GPU are based on 32-bit single precision computation of the fragment pipeline. For comparison, we have also implemented a software version of the LBM simulation using single precision floating point, and measured its performance on the same machine.

Figure 7 is a screen shot of our simulation system. View (a) shows the advected particles. View (b) shows the velocity texture. View (c) is a debugger window showing a Cg program. View (d) is a distribution texture. We could set break points or specify in the debugger window which variable to watch. Partial results of the computation are shown both visually and numerically in the texture views (b) and (d).

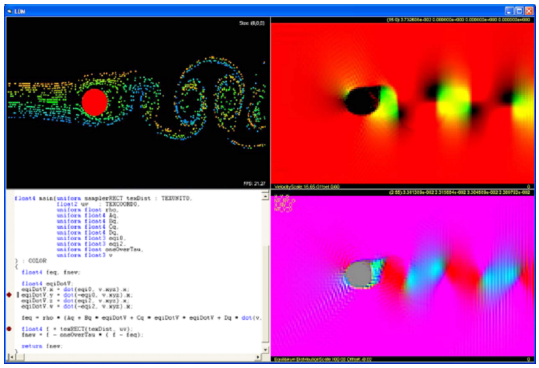


Figure 7: A screen shot of our simulation system: (a) simulation result; (b) velocity texture; (c) Cg program and debugger; (d) distribution texture $f1$.

Figure 8 shows a 2D flow field simulation based on the D2Q9 model with a lattice size of 256^2. We insert two circles (shown in red) into the field as the obstacles boundary conditions. The vortices are generated due to the obstacles. The figure only shows a snapshot of the flow at a certain moment. While the flow is relatively stable, yet it changes with time. Figure 1 shows 3D flow simulations using the D3Q19 model with a lattice size of 50^3. Note that our simulation can handle arbitrarily complex and dynamic boundary, which usually results in a complex flow field. We only show particles injected from a slit to reduce clutterness. The motion of the particles as well as the capability of changing the view angles help in understanding the shape of the flow. Simulation with more complex boundary conditions and more particles can be seen in the accompanying videos.

Figure 9 shows the time in milliseconds per step of the LBM D2Q9 model as a function of the lattice size, running on both the CPU and the GPU. Note that both the X and Y axes are in logarithmic scale. Figure 10 compares the two from a different perspective by showing the speedup factor. The times include both simulation

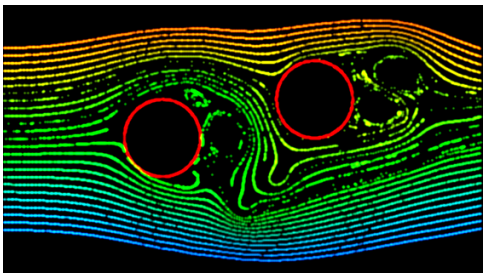


Figure 8: Particles advected in a 2D flow field based on the D2Q9 LBM model.

and visualization. Note that there is no need to transfer the velocity field or the particle positions between the main memory and the graphics memory. The time spent on advecting and rendering the particles is negligible with respect to the simulation.

For lattice size equals to or greater than 128^2, the simulation on the GPU is about 6 times faster than that on the CPU. The GPU is less effective in acceleration for smaller lattice sizes due to the overhead inside the GPU pipeline, such as switching textures and switching GL context that are independent of the lattice size. As the lattice size increases, the percentage of the overhead among the total time decreases, hence we see better speedups. For large lattice sizes, the overhead is negligible and the simulation times of both the GPU-based LBM and the CPU-based LBM tend to be proportional to the lattice size. Therefore, the speedup factor stays relatively constant.

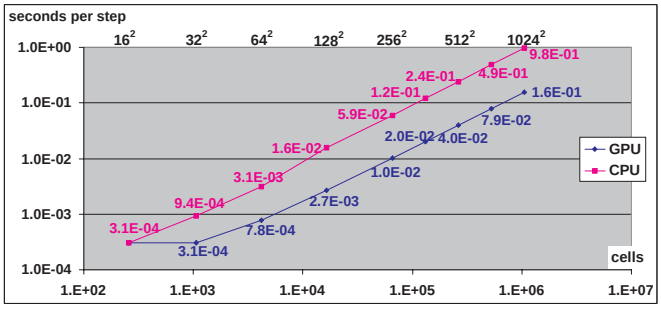


Figure 9: Time (milliseconds) per step of a D2Q9 LBM simulation.

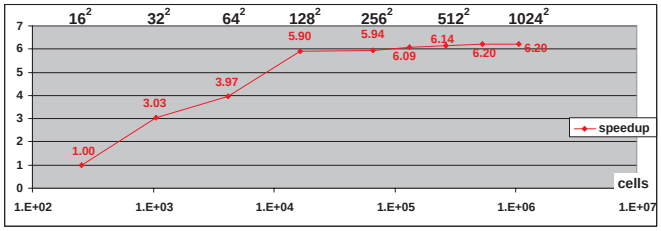


Figure 10: Speedup of the LBM on the GPU vs. the software version with the D2Q9 model.

Figures 11 and 12 show similar graphs to Figures 9 and 10, but for the D3Q19 LBM model. Obviously, the GPU version is faster than its software counterpart. The speedup factor varies between 8 and 9 for most of the lattice sizes. When the lattice size approaches 128^3, the speedup is as high as 15.

8 Conclusions

We have presented a fluid flow simulation using the physically-based LBM, accelerated by programmable graphics hardware. The LBM simulation and its boundary conditions are of second-order

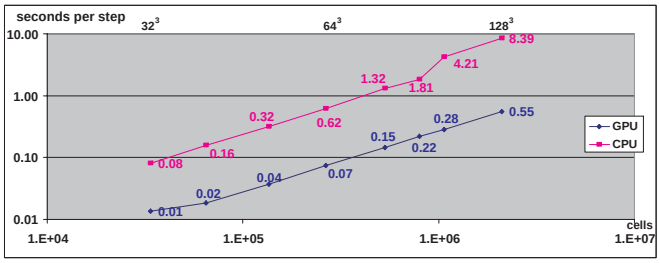


Figure 11: Time (seconds) per step of a D3Q19 LBM simulation.

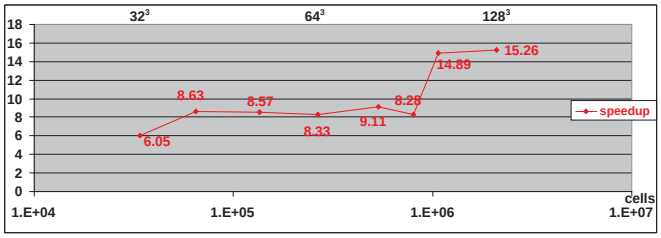


Figure 12: Speedup of the LBM on the GPU vs. the software version with the D3Q19 model.

accuracy in both time and space. Our GPU-based simulation using floating point computation achieves the same accuracy as the CPU-based LBM simulation, yet, it is much faster. Our experimental results have shown that the GPU version is significantly faster (e.g., 8-15 times for the D3Q19 model), for most of the lattice sizes in both 2D and 3D simulations. To attain these speeds, we have incorporated several optimization techniques into the GPU method. We have also proposed a GPU-based general-purpose voxelization algorithm and its extension to handle arbitrarily complex and dynamic boundary conditions in real time.

Although we only discuss the LBM in this paper, most, if not all, of our techniques apply to other models, particularly models defined on regular grids, such as Cellular Automata, explicit finite differences, and Coupled Map Lattice. We are currently working on a framework to unify these different models and are developing an automatic optimization toolkit for generating faster GPU programs. As our approach is scalable, we also plan to extend our technique to GPU clusters, which can provide large scale flow simulations to computational scientists.

Acknowledgement

The model and the rendering of the jellyfish are adapted from Nvidia's sample code.

References

BOLZ, J., FARMER, I., GRINSPUN, E., AND SCHRÖDER, P. 2003. Sparse matrix solvers on the GPU: Conjugate gradients and multigrid. *SIGGRAPH*.

CHEN, S., AND DOOLEAN, G. D. 1998. Lattice Boltzmann method for fluid flows. *Annu. Rev. Fluid Mech. 30*, 329–364.

EVERITT, C., 2001. Interactive order-independent transparency.

FANG, S., AND CHEN, H. 2000. Hardware accelerated voxelization. *Computers & Graphics 24*, 3 (June), 433–442.

FEDKIW, R., STAM, J., AND JENSEN, H. W. 2001. Visual simulation of smoke. *SIGGRAPH* (August), 15–22.

FOSTER, N., AND METAXAS, D. 1997. Modeling the motion of a hot, turbulent gas. *SIGGRAPH* (August), 181–188.

GOODNIGHT, N., WOOLLEY, C., LEWIN, G., LUEBKE, D., AND HUMPHREYS, G. 2003. A multigrid solver for boundary value probelms using programmable graphics hardware. *SIGGRAPH / Eurographics Workshop on Graphics Hardware*.

HARRIS, M., COOMBE, G., SCHEUERMANN, T., AND LASTRA, A. 2002. Physically-based visual simulation on graphics hardware. *SIGGRAPH / Eurographics Workshop on Graphics Hardware* (Sep), 109–118.

HARRIS, M., III, W. V. B., SCHEUERMANN, T., AND LASTRA, A. 2003. Simulation of cloud dynamics on graphics hardware. *SIGGRAPH / Eurographics Workshop on Graphics Hardware* (July).

HOFF, K., CULVER, T., KEYSER, J., LIN, M., AND MANOCHA, D. 1999. Fast computation of generalized voronoi diagrams using graphics hardware. *SIGGRAPH* (August), 277–286.

JOBARD, B., ERLEBACHER, G., AND HUSSAINI, M. Y. 2000. Hardware-accelerated texture advection for unsteady flow visualization. *IEEE Visualization* (October), 155–162.

KANDHAI, B. D. 1999. *Large Scale Lattice-Boltzmann Simulations.* PhD thesis, University of Amsterdam.

KRÜGER, J., AND WESTERMANN, R. 2003. Linear algebra operators for gpu implementation of numerical algorithms. *SIGGRAPH*.

LARSEN, E. S., AND MCALLISTER, D. 2001. Fast matrix multiplies using graphics hardware. *The International Conference for High Performance Computing and Communications*.

LEFOHN, A., KNISS, J., HANSEN, C., AND WHITAKER, R. 2003. Interactive deformation and visualization of level set surfaces using graphics hardware. *IEEE Visualization* (October), 75–82.

LI, W., WEI, X., AND KAUFMAN, A. 2003. Implementing lattice boltzmann computation on graphics hardware. *The Visual Computer 19*, 7-8 (December), 444–456.

MARK, W., GLANVILLE, S., AKELEY, K., AND KILGARD, M. 2003. Cg: A system for programming graphics hardware in a C-like language. *SIGGRAPH*.

MEI, R., LUO, L.-S., AND SHYY, W. 1999. An accurate curved boundary treatment in the lattice boltzmann method. *Journal of Comp. Phys. 155*, 307–330.

MEI, R., SHYY, W., YU, D., AND LUO, L.-S. 2000. Lattice Boltzmann method for 3-d flows with curved boundary. *Journal of Comp. Phys. 161* (March), 680–699.

PEERCY, M. S., OLANO, M., AIREY, J., AND UNGAR, P. J. 2000. Interactive multi-pass programmable shading. *SIGGRAPH* (July), 425–432.

PROUDFOOT, K., MARK, W. R., TZVETKOV, S., AND HANRAHAN, P. 2001. A real-time procedural shading system for programmable graphics hardware. *SIGGRAPH* (August), 159–170.

PURCELL, T., BUCK, I., MARK, W., AND HANRAHAN, P. 2002. Ray tracing on programable hardware. *SIGGRAPH* (August).

SHADE, J., GORTLER, S. J., WEI HE, L., AND SZELISKI, R. 1998. Layered depth images. *SIGGRAPH* (July), 231–242.

STAM, J. 1999. Stable fluids. *SIGGRAPH* (August), 121–128.

TRENDALL, C., AND STEWART, A. J. 2000. General calculations using graphics hardware with applications to interactive caustics. *Eurographics Workshop on Rendering* (June), 287–298.

WEI, X., LI, W., MUELLER, K., AND KAUFMAN, A. 2002. Simulating fire with texture splats. *IEEE Visualization*, 227–234.

WEI, X., LI, W., MUELLER, K., AND KAUFMAN, A. 2004. Lattice-boltzmann method for amorphous phenomena. *IEEE Transaction on Visualization and Computer Graphics 10*, 2 (March/April), 164–176.

WEISKOPF, D., HOPF, M., AND ERTL, T. 2001. Hardware-accelerated visualization of time-varying 2D and 3D vector fields by texture advection via programmable per-pixel operations. *Workshop on Vision, Modeling, and Visualization VMV*, 439–446.

WOLF-GLADROW, D. A. 2000. *Lattice gas cellular automata and lattice Boltzmann models: an introduction.* Springer-Verlag.